

The background features a light blue gradient with various geometric shapes. Numerous triangles in different shades of blue (light, medium, and dark) are scattered across the frame. Some triangles are solid, while others are outlined or semi-transparent. Dotted lines connect some of the triangles, creating a network-like pattern. The overall aesthetic is modern and tech-oriented.

價值投資 智慧選股系統

AIPE03 · 01 · 曹品誠

前言

為何想做這個內容

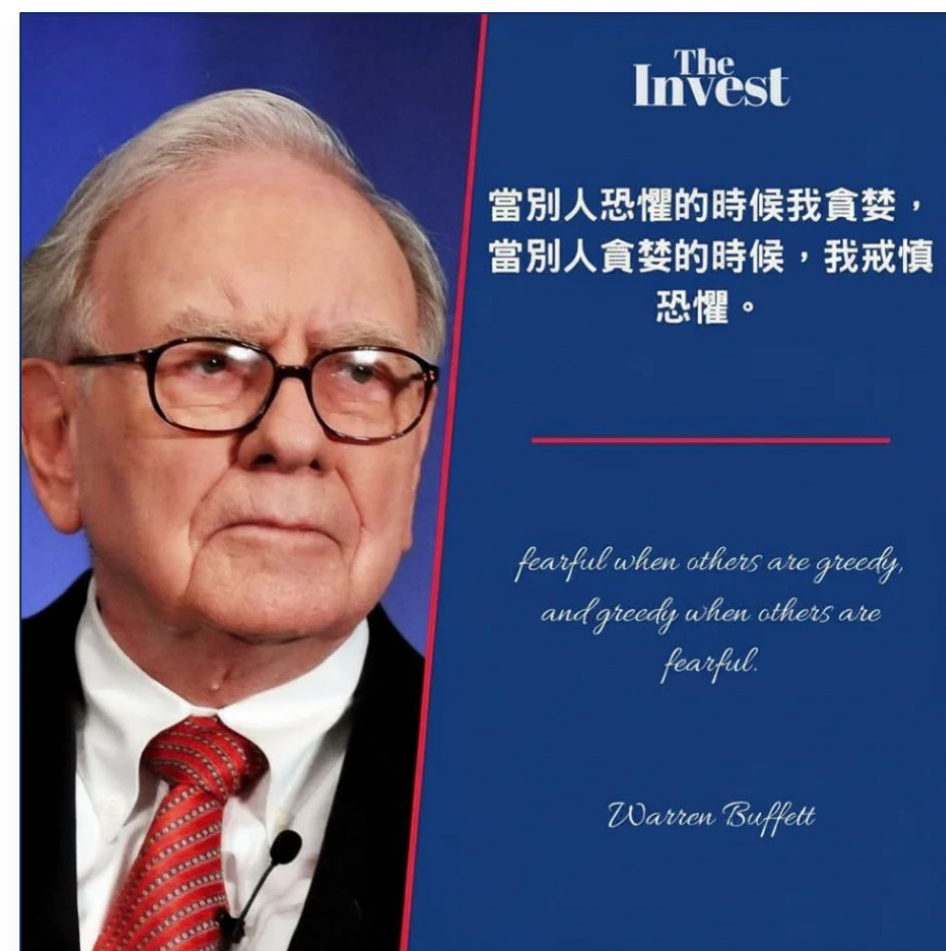
01

股市受益者



02

巴菲特的價值投資



03

聽股票專家千話 不如自己親眼所見 一行數據



ROE · 股東權益報酬率

系統架構

01

DATA SOURCING

資料來源層

- › 技術線:Selenium / Requests
- › 目標:向公開資訊觀測站(MOPS)動態抓取財報
- › 機制:自動重置 jsession,防止 IP 被 MOPS 封鎖

02

DATA PARSING

解析清洗層

- › 技術線:Pandas
- › 目標:將非標準 HTML 原始碼轉為結構化 DataFrame
- › 函數:parse_mops_html / flatten_cols / find_col

03

METRICS CALCULATION

指標計算層

- › 技術線:pandas 的 merge / apply / groupby
- › 目標:自動衍生關鍵財務與籌碼指標
- › 指標:ROE、毛利率、自由現金流(FCF)、內部人持股比

04

STRATEGY FILTERING

策略篩選層

- › 目標:透過 5 大核心量化條件,實施多重過濾漏斗
- › 條件:EPS 連續正值 · 利潤率趨勢成長 · 負債比 < 50%
- › ROE > 10% · 連續配息

05

OUTPUT & DISPLAY

結果輸出層

- › 目標:支援多終端格式,因應不同場景提取
- › 格式:CSV 篩選結果 / mops_data.pkl / Streamlit App

公開資訊觀測站 (Market Observation Post System, 簡稱 MOPS)

爬取資料來源

01 LEGAL AUTHORITY · 

具備國家法規的最高法律效力

視同正式公告 — 依金管會證券法規,向 MOPS 申報傳輸成功即視為依規定完成公告申報。

法律追訴權 — 虛偽不實或隱匿者,董事長與高階須負《證券交易法》刑事責任。

02 REAL-TIME · 

絕對的第一手即時性

重訊即時防線 — 重大事件須於指定時限(通常次一營業日開盤前)在 MOPS 刊登。

嚴防內線交易 — 散戶、外資、投信、記者同一時間點看到資訊,維持市場公平。

03 RAW DATA · 

經會計師簽證審核的原始數字

拒絕二手加工 — 看盤軟體與財經網站多經過二次重組,有工程師寫錯邏輯的 Bug 風險。

黃金標準 (Gold Standard) — MOPS 提供會計師簽證的原始財報(資產負債、損益、現金流)。

04 OFFICIAL DISCLOSURE · 

企業官方的防造謠 / 澄清專區

唯一官方正名管道 — 媒體報導未經證實時,金管會強制公司於當天至 MOPS 重訊區澄清。

官方重訊為準 — 市場謠言不作數,一切以 MOPS 上的官方重大訊息為依據。

資料來源聲明

1. 本教學 / 研究展示所使用之財務數據與企業資訊,均公開引述自臺灣證券交易所「公開資訊觀測站」。
2. 本範例及相關程式碼僅供學術研究與教學演示使用 (Non-commercial Educational Use Only),未涉及任何商業販售或營利行為。

Python 模組引用

```
import pickle, subprocess, sys, time, random, webbrowser
import requests
import pandas as pd
import numpy as np
from io import StringIO
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select, WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.chrome import ChromeDriverManager
```

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import pickle
```


取關鍵 Cookie

- 01

偽裝準備 (Options)

抹除機器人特徵 · 填入真人 User-Agent · 開啟無痕模式
- 02

登入潛入 (Selenium)

開啟瀏覽器進入 MOPS,騙過檢測並取得 jcsession 金鑰
- 03

偷取金鑰 (Cookie Theft)

建立 requests.Session() · 把瀏覽器 Cookies 全部複製過去
- 04

安全撤退 (Clean Up)

關閉 Selenium 瀏覽器(省記憶體) · 留下帶金鑰的 Session 備用

```
# 強制覆寫 Base URL (舊版 mopsov 才有 jcsession; 新版 mops.twse.com.tw 會封鎖)
MOPS_BASE = 'https://mopsov.twse.com.tw/mops/web'

def create_session(headless: bool = False) -> requests.Session:
    opts = Options()

    # — 防偵測設定 —
    opts.add_argument('--disable-blink-features=AutomationControlled')
    opts.add_argument(
        '--user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) '
        'AppleWebKit/537.36 (KHTML, like Gecko) '
        'Chrome/143.0.0 Safari/537.36'
    )
    opts.add_argument('--incognito')

    # — 視窗模式 —
    if headless:
        opts.add_argument('--headless=new') # 新版 headless (較難被偵測)
        opts.add_argument('--window-size=1280,800')
    else:
        opts.add_argument('--start-maximized')

    # — 其他穩定性設定 —
    opts.add_argument('--no-sandbox')
    opts.add_argument('--disable-dev-shm-usage')
    opts.add_argument('--disable-gpu')

    # 移除 WebDriver 特徵旗標
    opts.add_experimental_option('excludeSwitches', ['enable-automation'])
    opts.add_experimental_option('useAutomationExtension', False)

    driver = webdriver.Chrome(
        service=Service(ChromeDriverManager().install()),
        options=opts
    )

    # 進一步隱藏 navigator.webdriver
    driver.execute_cdp_cmd(
        'Page.addScriptToEvaluateOnNewDocument',
        {'source': 'Object.defineProperty(navigator, webdriver, {get: () => undefined})'}
    )

    landing = MOPS_BASE + '/t163sb04'
    print(f'Visiting: {landing} ...', end=' ', flush=True)

    try:
        driver.get(landing)
        time.sleep(3)
        print(f'title: {driver.title[:40]}')

        sess = requests.Session()
        for c in driver.get_cookies():
            sess.cookies.set(c['name'], c['value'])

        cookie_names = list(sess.cookies.keys())
        print(f'Cookies: {cookie_names}')

        if 'jcsession' not in cookie_names:
            raise RuntimeError(
                'jcsession 未取得! 請確認可連線至 mopsov.twse.com.tw'
            )

        print('Session ready.')
        return sess

    finally:
        driver.quit() # 無論成功或失敗都關閉瀏覽器

# headless=False 開啟可見瀏覽器; 改 True 可背景執行
SESSION = create_session(headless=False)
```

資料抓取、清洗

(損益表、資產負債表、現金流量表)

01 `sess.post/ajax_url)` ← requests 發送 POST

02 `r.text` HTML 字串

03 `parse_mops_html(r.text, ...)` 自訂解析函數

04 `pd.read_html(StringIO(html))` ← pandas 解析 HTML 表格

05 `max(tables, key=len)` ← 取最大表格

06 `DataFrame` 結構化結果

```
time.sleep(random.uniform(3, 7))

IS AJAX = MOPS_BASE + '/ajax_t163sb04'
IS_PAGE = MOPS_BASE + '/t163sb04'

def fetch_income_stmt(sess, year, season='04', typek=TYPEK):
    payload = {
        'encodeURIComponent': '1', 'step': '1',
        'firstin': 'true', 'off': '1',
        'TYPEK': typek, 'year': str(year), 'season': season,
    }
    hdrs = {**BASE_HDR, 'Referer': IS_PAGE}
    try:
        r = sess.post(IS AJAX, data=payload, headers=hdrs, timeout=40)
        r.encoding = 'utf-8'
        if not r.text.strip():
            print(f' {year}: 空回應')
            return None
        if 'SECURITY REASONS' in r.text:
            print(f' {year}: 被封鎖 請重新執行 Cell 3')
            return None
        return parse_mops_html(r.text, year, season)
    except Exception as e:
        print(f' {year}: 錯誤 {e}')
        return None

print('=== 損益表彙總表 (t163sb04) ===')
is_frames = []
for yr in YEARS:
    print(f' {yr}...', end=' ', flush=True)
    df = fetch_income_stmt(SESSION, yr)
    if df is not None and not df.empty:
        is_frames.append(df)
        print(f'{len(df)} 家')
    else:
        print('無資料')
    time.sleep(1.5)

df_is_raw = pd.concat(is_frames, ignore_index=True) if is_frames else pd.DataFrame()
print(f'\n合計: {len(df_is_raw)} 筆')
if not df_is_raw.empty:
    print(f'涵蓋年度: {sorted(df_is_raw["year"].unique())}')
    print(f'欄位範例: {list(df_is_raw.columns[:5])}')
```


Selenium 運用

(股利分派)

- 01

建立瀏覽器 + 進入頁面

make_div_driver / driver.get · 等待表單元素就緒
- 02

填寫條件

年份 · 市場類型(上市) · qryType radio
- 03

點擊查詢

JS click 觸發 · 記錄點擊前視窗清單
- 04

等待 Popup

number_of_windows_to_be(+1) · 最多等 20 秒
- 05

切換視窗

switch_to.window(popup_handle) · 等 table 出現
- 06

取 HTML

driver.page_source · 關閉 popup · 切回主視窗
- 07

解析表格

pd.read_html() · 過濾欄位 < 15 的雜表 · 保留公司代號列

```
time.sleep(random.uniform(3, 7))

DIV_URL = MOPS_BASE + '/t05st09_new'

def make_div_driver():
    opts = Options()
    opts.add_argument('--headless=new')
    opts.add_argument('--no-sandbox')
    opts.add_argument('--disable-dev-shm-usage')
    opts.add_argument('--disable-gpu')
    opts.add_argument('--disable-blink-features=AutomationControlled')
    opts.add_argument('--disable-popup-blocking')
    opts.add_argument('--window-size=1280,900')
    opts.add_argument(
        '--user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) '
        'AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36'
    )
    opts.add_experimental_option('excludeSwitches', ['enable-automation'])
    opts.add_experimental_option('useAutomationExtension', False)
    opts.add_experimental_option('prefs', {
        'profile.default_content_setting_values.popups': 1
    })
    d = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=opts)
    d.execute_cdp_cmd('Page.addScriptToEvaluateOnNewDocument',
        [{ 'source': 'Object.defineProperty(navigator, "webdriver", {get:()=>undefined})' }])
    d.set_page_load_timeout(30)
    d.set_script_timeout(15)
    return d

def get_popup_html(driver, year, typek, qrytype):
    driver.get(DIV_URL)

    # 等待頁面核心元素就緒
    try:
        WebDriverWait(driver, 15).until(
            EC.presence_of_element_located((By.NAME, 'year'))
        )
    except Exception:
        print(f'    {year} qryType={qrytype}: 頁面載入逾時', flush=True)
        return None

    print(f'\n    URL={driver.current_url}', flush=True)

    # 設定年份
    for inp in driver.find_elements(By.TAG_NAME, 'input'):
        if inp.get_attribute('name') == 'year':
            driver.execute_script('arguments[0].value=arguments[1];', inp, str(year))

    # 設定市場類型
    for sel in driver.find_elements(By.TAG_NAME, 'select'):
        if sel.get_attribute('name') == 'TYPEK':
            try:
                Select(sel).select_by_value(typek)
            except Exception:
                pass

    # 點選 qryType radio
    for inp in driver.find_elements(By.TAG_NAME, 'input'):
        if inp.get_attribute('name') == 'qryType' and inp.get_attribute('value') == qrytype:
            driver.execute_script('arguments[0].click();', inp)
            break

    # 找查詢按鈕
    try:
        btns = WebDriverWait(driver, 10).until(
            EC.presence_of_all_elements_located(
                (By.XPATH, '//input[@type="button" and contains(@value,"查詢")]')
            )
        )
    except Exception:
        print(f'    {year} qryType={qrytype}: 找不到查詢按鈕', flush=True)
        return None

    print(f'    查詢按鈕數量: {len(btns)}', flush=True)
```

```
handles_before = set(driver.window_handles)
main_handle = driver.current_window_handle
driver.execute_script('arguments[0].click();', btns[0])

# 等待新視窗開啟
try:
    WebDriverWait(driver, 20).until(
        EC.number_of_windows_to_be(len(handles_before) + 1)
    )
    popup_handle = (set(driver.window_handles) - handles_before).pop()
    driver.switch_to.window(popup_handle)

    # 等待 popup 內表格出現
    WebDriverWait(driver, 20).until(
        EC.presence_of_element_located((By.TAG_NAME, 'table'))
    )
    html = driver.page_source
    driver.close()
    driver.switch_to.window(main_handle)
    return html

except Exception as e:
    print(f'    彈窗等待失敗: {e}', flush=True)
    # fallback: 如果新視窗已開啟但條件未觸發
    newWins = set(driver.window_handles) - handles_before
    if newWins:
        driver.switch_to.window(newWins.pop())
        html = driver.page_source
        driver.close()
        driver.switch_to.window(main_handle)
        return html
    try:
        driver.switch_to.window(main_handle)
    except Exception:
        pass
    return None

def parse_div_html(html, year, qrytype):
    tables = pd.read_html(StringIO(html), flavor='lxml')
    frames = []

    for t in tables:
        if t.shape[1] < 15:
            continue
        t = flatten_cols(t.copy())
        first_col = t.columns[0]
        mask = t[first_col].astype(str).str.match(r'^\d{4}')
        t = t[mask].copy()
        if not t.empty:
            t['year'] = year
            t['qryType'] = qrytype
            frames.append(t)

    return pd.concat(frames, ignore_index=True) if frames else None

print('=== 股利分派情形 (t05st09_new) ===')
print(f'DIV_URL = {DIV_URL}')
print('建立 Selenium 驅動中...')
_div_driver = make_div_driver()

div_frames = []
for yr in YEARS:
    yr_frames = []
    for qt in ['1', '2']:
        print(f'\n    {yr} qryType={qt}...', end=' ', flush=True)
        html = get_popup_html(_div_driver, yr, TYPEK, qt)
        if html:
            df = parse_div_html(html, yr, qt)
            if df is not None:
                yr_frames.append(df)
                print(f'→ {len(df)} 列', end=' ')
            else:
                print('→ 解析失敗', end=' ')
        else:
            print('→ 無彈出視窗', end=' ')
        time.sleep(random.uniform(1, 2))
    if yr_frames:
        div_frames.append(pd.concat(yr_frames, ignore_index=True))

_div_driver.quit()
print('\n瀏覽器已關閉')
```


指標計算

PROFITABILITY

毛利率

毛利 ÷ 營業收入

PROFITABILITY

營業利益率

營業利益 ÷ 營業收入

FALLBACK · 備用算法

毛利

營業收入 - 營業成本

報表無毛利欄時啟用

LEVERAGE

負債比率

總負債 ÷ 總資產

EFFICIENCY

ROE 股東權益報酬率

本期淨利 ÷ 股東權益

CASH FLOW

FCF 自由現金流

營業現金流 + 投資活動現金流

≈ 營業現金流 - 資本支出

DIVIDEND CHECK · 配息檢查

是否配息



現金股利 > 0 → 有配息



現金股利 = 0 / 空白 → 未配息

NOTE

其他數據透過對應欄位直接取值。

策略篩選

01 EPS 連續正值
每年 EPS > 0, 資料 ≥ 5 年 · .all() 整組判斷

02 利潤率趨勢 (最複雜)
毛利率序列 vs [0, 1, 2, 3, 4] 皮爾森相關係數 ≥ -0.10
毛利率 + 營業利益率各算一次, 皆須通過

03 負債比率上限
每年負債比率 < 50% · 全部年度都要符合

04 ROE 下限
每年 ROE > 10% · 全部年度都要符合

05 連續配息年數
有配息的年度數量 ≥ 5 年

06 最終交集 候選名單
EPS ∩ 趨勢 ∩ 負債 ∩ ROE ∩ 配息

```
# 1. EPS > 0 連續 MIN_EPS_YEARS 年
def check_eps(g):
    return (g['eps'] > 0).all() and len(g) >= MIN_EPS_YEARS

pass_eps = set(
    df_is.groupby('id').filter(check_eps)['id'].unique()
) if not df_is.empty else set()

# 2. 毛利率 & 營業利益率: 持平或逐年走高 (相關係數 >= -0.1)
def is_stable_or_rising(series):
    vals = series.dropna()
    if len(vals) < 2:
        return False
    corr = pd.Series(vals.values).corr(pd.Series(range(len(vals))))
    return pd.notna(corr) and corr >= MARGIN_TREND_CORR

def check_margin(g):
    g_s = g.sort_values('year')
    return (
        is_stable_or_rising(g_s['gross_margin']) and
        is_stable_or_rising(g_s['op_margin'])
    )

pass_margin = set(
    df_is.groupby('id').filter(check_margin)['id'].unique()
) if not df_is.empty else set()

# 3. 負債比率 < DEBT_THRESHOLD 每年
def check_debt(g):
    vals = g['debt_ratio'].dropna()
    return len(vals) > 0 and (vals < DEBT_THRESHOLD).all()

pass_debt = set(
    df_bs.groupby('id').filter(check_debt)['id'].unique()
) if not df_bs.empty else set()

# 4. ROE > ROE_THRESHOLD 每年
def check_roe(g):
    if 'roe' not in g.columns: return False
    vals = g['roe'].dropna()
    return len(vals) > 0 and (vals > ROE_THRESHOLD).all()

pass_roe = set(
    df_bs.groupby('id').filter(check_roe)['id'].unique()
) if (not df_bs.empty and 'roe' in df_bs.columns) else set()

# 5. 連續 MIN_DIV_YEARS 年有現金配息
if not df_div.empty:
    div_years = (
        df_div[df_div['has_dividend']]
        .groupby('id')['year'].nunique()
    )
    pass_div = set(div_years[div_years >= MIN_DIV_YEARS].index)
else:
    pass_div = set()

candidates = pass_eps & pass_margin & pass_debt & pass_roe & pass_div
```


結果輸出

01 啟動

Cell 12 背景執行 Streamlit · 開啟 localhost:8501

02 載入資料

讀取 mops_data.pkl · 失敗則改讀 mops_master.csv

03 使用者選擇

Sidebar 選公司範圍 · 拖拉年度 slider

04 即時過濾

flt() 依選擇切出對應公司 × 年度的資料子集

05 圖表顯示

五個 Tab 自動重繪 · 損益 / 資負 / 股利 / 現金流 / 篩選總覽

```
# 背景啟動 Streamlit
_proc = subprocess.Popen(
    [sys.executable, '-m', 'streamlit', 'run', 'stock_app.py',
     '--server.port', '8501',
     '--server.headless', 'true',
     '--browser.gatherUsageStats', 'false'],
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL,
)

time.sleep(3) # 等待服務啟動

URL = 'http://localhost:8501'
print('=' * 50)
print(f' Streamlit 已啟動:{URL} ')
print('=' * 50)
print()
print('  操作說明:')
print('    左側 Sidebar → 選擇公司、年度範圍')
print('    Tab 1 篩選總覽 → 結果表 + ROE/負債散點圖')
print('    Tab 2 損益趨勢 → EPS、毛利率、營業利益率折線圖')
print('    Tab 3 資產負債 → 負債比、ROE、資產結構')
print('    Tab 4 股利分析 → 配息金額、配息熱力圖')
print()
print('  停止服務請執行: _proc.terminate()')

# 自動在瀏覽器開啟
try:
    webbrowser.open(URL)
except Exception:
    pass
```

遇到問題

三個踩過的坑與解法

PROBLEM 01

新版網站 WAF 防火牆 會鎖 IP

SOLUTION

改連舊版網站 `mopsov.twse.com.tw`

透過 Selenium 取得 `jcsession` 後再爬取。

PROBLEM 02

表格欄位對不上 資料很髒

SOLUTION · 三步清洗

① 壓平多層標頭 — 兩層標頭用空格串接成單字串

② 關鍵字掃描 — 欄位名含關鍵字即視為找到

③ 數字清洗

```
"1,234" → 1234 (500) → -500 str → float()
```

PROBLEM 03

爬取時 回傳空列表

SOLUTION · 先看再判

先直接開啟網站確認狀況：

鎖 IP → 等待 / 換 IP

彈窗擋住 → Selenium 模擬點擊取得 HTML 在結構化

使用工具



主戰場 · MAIN

VS Code

編輯器 / Debug



主戰場 · MAIN

Python

爬蟲 / 分析語言



觀念釐清 · 科普

Google AI

搜尋 AI 模式



技術診察 · 支援

Claude

debug / 找 bug

未來展望

把篩選邏輯延伸到其他市場 · 邁向完整交易系統

TARGET MARKETS

虛擬貨幣 · Crypto

美股 · US Stocks

STAGE · 01

加入價格資料

把現有「體質篩選」連接到日線 / 即時報價,讓基本面結合行情。

PRICE DATA



STAGE · 02

量化策略

用 Python 把訊號量化:進場 / 出場條件、權重配置、停損停利規則。

STRATEGY



STAGE · 03

歷史回測

用過去 5–10 年資料驗證策略表現:報酬、回撤、夏普值。

BACKTEST



STAGE · 04

自動化交易

串接券商 / 交易所 API · 排程執行 · 持續監控與通知。

AUTOMATION

名言分享

不要追逐利潤,讓利潤追著你跑。 — 經營之神 稻盛和夫

Thank You

